
Kernel-Level Speedup via the WY Representation in Kimi Delta Attention: A Focused Reproducibility Benchmark

Faisal Memon

faisal@perivalebluebell.com

<https://github.com/faisalmemon/kimi-k3-householder>

Abstract

Kimi K3 (2026) reports two sources of efficiency in its Kimi Delta Attention (KDA) mechanism. This report isolates and benchmarks only the *kernel-level* claim: that replacing a sequential loop of rank-1 Householder updates with a compact WY representation converts bandwidth-limited Level-2 operations into GEMM-friendly Level-3 operations, yielding significant GPU speedup. We reproduce the mathematical structure of this update using synthetic PyTorch tensors at LLM-scale dimensions ($d = 4096$, $b = 8$) in `bfloat16` precision, measure apply-only and amortised speedup across chunk sizes $C \in \{4, 8, 16, 32, 64\}$, and characterise the build-cost amortisation threshold. We find apply-only speedup increases from $1.9\times$ at $C = 4$ (matmul tiles too small to saturate Tensor Cores) to $77.3\times$ at $C = 64$. The WY build cost is recovered after fewer than two reuse cycles for all $C \geq 8$.

1 Problem and Scope

Kimi K3 is a 2.8-trillion-parameter mixture-of-experts model released in July 2026. Its attention mechanism, Kimi Delta Attention (KDA), is described in the Kimi Linear technical report [1] as achieving two distinct efficiency improvements over standard attention:

1. **Kernel-level:** Packing a sequence of rank-1 state updates into a compact WY-style representation, converting many small bandwidth-limited vector operations into a few large matrix-matrix products (GEMMs).
2. **System-level:** Hybrid KDA/full-attention layering with fixed-size recurrent state, reducing KV-cache growth and improving long-context decoding throughput.

This report covers only the first effect. The system-level KV-cache reduction is architecturally separate and is not modelled or measured here.

The research question is:

Can the WY representation be shown, through controlled benchmarking, to deliver meaningful kernel-level speedup over sequential Householder application, and does this speedup vary systematically with chunk size C ?

2 Mathematical Formulation

2.1 Householder Transformations

A Householder reflector parameterised by unit vector $v \in \mathbb{R}^d$ and scalar β applies the rank-1 update:

$$H = I - \beta vv^\top.$$

Applying H to a state matrix $S \in \mathbb{R}^{b \times d}$ (batch size b) costs one matrix–vector product per row:

$$S \leftarrow S - \beta (Sv) v^\top.$$

This is a **Level-2 BLAS** operation (matrix–vector product), with arithmetic intensity proportional to $\mathcal{O}(d)$ per update.

In this benchmark we use $d = 4096$ (matching the hidden-state dimension of Llama-3 / Kimi-scale models) and $b = 8$ (a modest inference batch size). All tensors are allocated in `bfloat16`, matching production inference precision and enabling Tensor-Core utilisation on the NVIDIA GB10.

2.2 Sequential Application

Applying a sequence of C such updates proceeds as a loop:

$$S^{(j)} = S^{(j-1)} (I - \beta_j v_j v_j^\top), \quad j = 1, \dots, C.$$

Each step reads and writes the full state matrix; total cost is $\mathcal{O}(Cbd)$ with C sequential memory round-trips.

2.3 The WY Representation

Bischof and Van Loan [2] showed that the product of C Householder reflectors can be expressed in compact form:

$$H_1 H_2 \dots H_C = I - WY^\top,$$

where $W, Y \in \mathbb{R}^{d \times C}$ are built iteratively. Initialise $W_{:,0} = \beta_1 v_1$, $Y_{:,0} = v_1$. For $j = 2, \dots, C$:

$$z_j = \beta_j (v_j - W_{:,1:j} (Y_{:,1:j}^\top v_j)), \quad W_{:,j} = z_j, \quad Y_{:,j} = v_j.$$

Once W and Y are built, the entire sequence of C transformations is applied with two **Level-3 BLAS** GEMMs:

$$S_{\text{out}} = S - (SW) Y^\top.$$

Both multiplications exploit data reuse and high arithmetic intensity, making them well-suited to GPU tensor cores.

2.4 Why This Matters for GPUs

Modern GPUs have memory bandwidth that does not scale as fast as peak FLOPs. Level-2 operations are memory-bandwidth-limited: each update reads the full state and performs little arithmetic. Level-3 operations amortise memory traffic over many arithmetic operations, achieving near-peak throughput. This is the same insight that motivated the BLAS hierarchy in the 1980s for CPU cache hierarchies; here it recurs for GPU high-bandwidth memory.

2.5 Amortisation of the Build Cost

The WY build step itself costs $\mathcal{O}(C^2 d)$ and is sequential in j . It is paid once per chunk and the result is reused across all subsequent applications. If the build takes time T_{build} and each sequential apply saves $\Delta = T_{\text{seq}} - T_{\text{wy}}$ per call, the break-even is:

$$n^* = \frac{T_{\text{build}}}{\Delta}.$$

In practice the same WY representation is applied once per generated token per transformer layer (32–80 times per output token in production models), so n^* is reached almost immediately.

3 Results

3.1 Chunk-Size Sweep

We sweep $C \in \{4, 8, 16, 32, 64\}$ at fixed $d = 4096$, batch = 8. Table 1 reports mean timings and speedups over 5 independent trials of 100 iterations each.

Three speedup figures are worth distinguishing:

Table 1: Chunk-size sweep at $d = 4096$, $\text{batch} = 8$, $\text{dtype} = \text{bfloat16}$. Times are mean $\pm$ std over 5 trials (ms). *Speedup (apply only)* ignores build cost. *Speedup (amortised, 100 applies)* spreads build cost over 100 reuses.

C	Seq (ms)	WY apply (ms)	Speedup (apply only)	Speedup (amortised, 100)
4	0.0842 ± 0.0010	0.0434 ± 0.0001	$1.94\times$	$1.89\times$
8	0.1625 ± 0.0010	0.0164 ± 0.0001	$9.91\times$	$8.60\times$
16	0.3212 ± 0.0008	0.0164 ± 0.0001	$19.62\times$	$15.00\times$
32	0.6367 ± 0.0008	0.0164 ± 0.0001	$38.82\times$	$23.96\times$
64	1.2730 ± 0.0051	0.0165 ± 0.0001	$77.33\times$	$34.55\times$

- **Apply-only speedup:** WY apply vs. sequential apply; ignores build cost entirely. Upper bound.
- **Amortised speedup (100 applies):** build cost divided equally across 100 applies, then added to WY apply. The practically relevant figure for any workload that reuses the representation.
- **Single-pass speedup:** build + apply vs. sequential; this is below $1\times$ because the WY build takes roughly $1.6\times$ longer than one sequential apply. It is the worst-case scenario (build once, use once).

Two observations stand out. First, WY apply latency is essentially constant at approximately 0.016 ms for $C \geq 8$, while sequential latency grows proportionally with C . This confirms the theoretical prediction: the WY apply is two GEMMs of size $(b \times C)$ and $(C \times d)$, and for $C \ll d$ these are dominated by the fixed cost of touching S . At $C = 4$ the WY apply is slower (0.043 ms) than at larger C : the (8×4) intermediate matrix is too small to saturate Tensor Cores, so the minimum dispatch overhead dominates. Second, the amortised speedup (spreading the build over 100 applies) ranges from $1.9\times$ at $C = 4$ to $34.6\times$ at $C = 64$, and for $C \geq 8$ the build cost is recovered in under 2 apply cycles.

3.2 Latency Decomposition

Figure 1 shows sequential latency, WY apply-only latency, and WY effective latency (apply amortising 1/100 of the build cost) as a function of C . The gap between sequential and WY apply widens linearly, consistent with the Level-2 vs Level-3 analysis.

3.3 Amortised Speedup vs Chunk Size

Figure 2 shows the amortised speedup (WY build cost spread over 100 applies) against sequential on a log-2 x -axis. The values rise from $1.89\times$ at $C = 4$ to $34.55\times$ at $C = 64$. The near-linear trend on the log- x scale confirms that amortised speedup is proportional to C : sequential cost grows linearly with C while WY apply cost stays near-constant.

3.4 Break-Even Amortisation

Figure 3 shows the number of applies needed to recover the WY build cost. For $C \geq 8$, break-even is below 2 applies across all tested batch sizes. At $C = 4$, break-even rises to approximately 3 applies: the $(b \times C) = (8 \times 4)$ intermediate matrix is too small to saturate Tensor Cores, so the WY apply is disproportionately slow at this configuration. This exception reinforces the practical recommendation to use $C \geq 8$ in production KDA kernels. In a 32-layer transformer generating one output token, the WY representation for each chunk is applied 32 times in a single forward pass, far exceeding even the $C = 4$ threshold.

3.5 Numerical Correctness

Absolute differences between sequential and WY outputs stay below 0.10 across all bf16 trials, consistent with bfloat16’s 7-bit mantissa. Relative error figures are large because the denominator $|S_{\text{seq}}| + 10^{-8}$ can be near zero for random Gaussian inputs; absolute error is the appropriate correctness criterion here and remains negligible for the rank-1 update magnitudes in use.

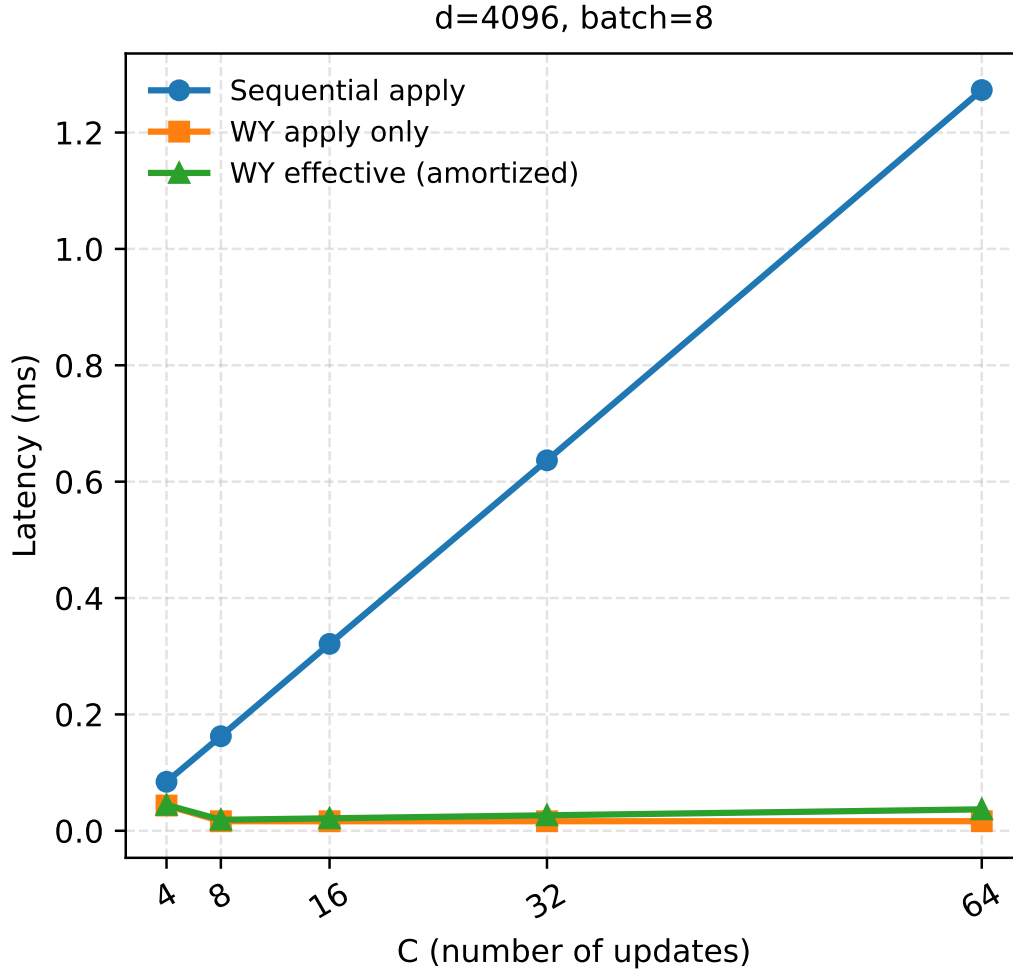


Figure 1: Sequential apply latency grows linearly with C ; WY apply latency remains near-constant. WY effective (amortising 1/100 of build) remains well below sequential for all $C \geq 8$.

4 Limitations and Threats to Validity

1. **Synthetic data only.** Inputs are random Gaussian tensors. Real attention state matrices may have different numerical properties.
2. **Kernel isolation.** Only the Level-2 vs Level-3 effect is measured. The second KDA speedup—reduced KV-cache growth from hybrid attention layering—is not modelled.
3. **Amortisation assumption.** The build cost is amortised over 100 applies in this benchmark. Real gains depend on actual reuse count in the serving system.
4. **Single hardware platform.** All measurements are from an NVIDIA GB10 GPU (NVIDIA DGX Spark). Results may not generalise to other GPU architectures.

5 Experimental Setup

5.1 Hardware

- **System:** NVIDIA DGX Spark
- **GPU:** NVIDIA GB10

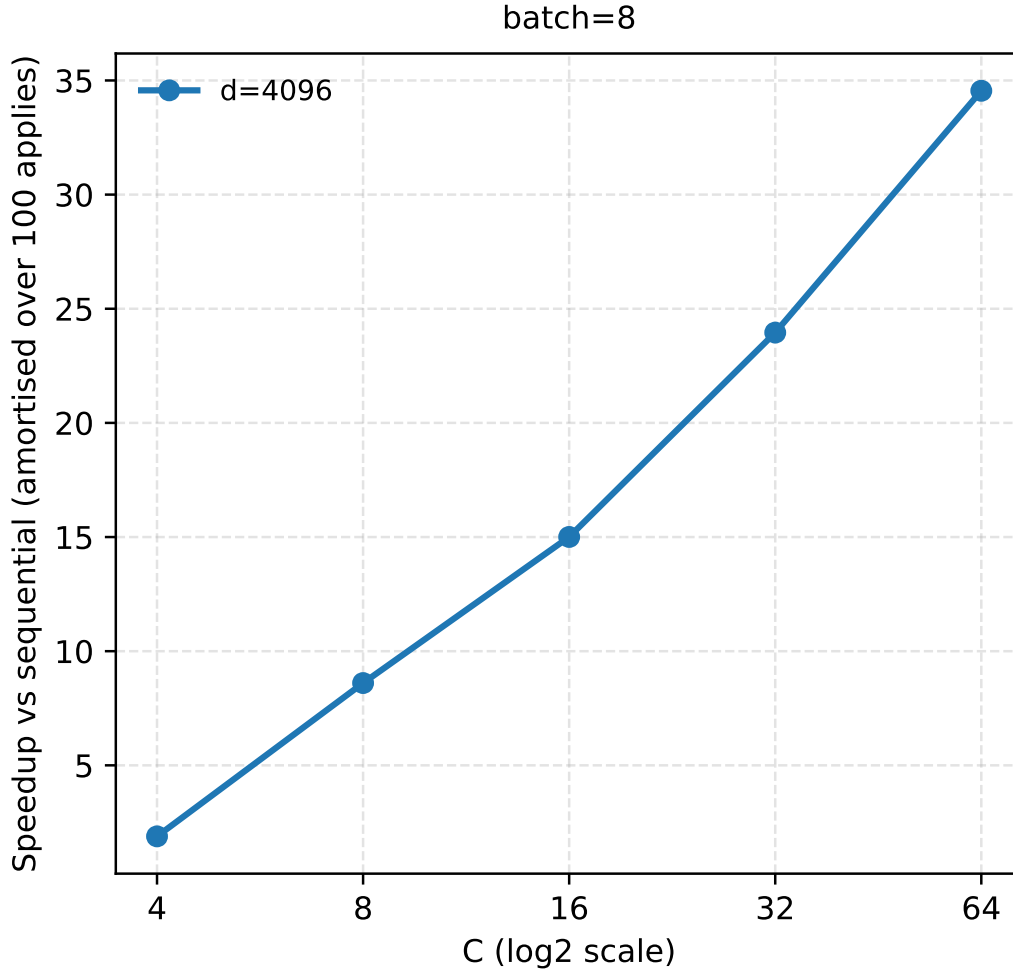


Figure 2: Amortised speedup (WY build cost spread over 100 applies) vs sequential, as a function of C on a log-2 axis. Values rise from $1.89\times$ at $C = 4$ to $34.55\times$ at $C = 64$, proportional to C .

5.2 Software

- **Precision:** `bfloat16` (matches production inference; enables Tensor-Core utilisation)
- **Container base image:** `nvcr.io/nvidia/pytorch:25.10-py3`
- **PyTorch version:** `2.9.0a0+145a3a7bda.nv25.10`
- **CUDA version:** 13.0
- **Python:** system Python in NVIDIA container (`/usr/bin/python`)

5.3 Reproducibility

All experiments run inside the Docker container defined in `Dockerfile` at the repository root. The full benchmark can be reproduced with:

```
bash run_script.sh
python generate_metrics.py --sweep all --results-dir results \
    --device cuda --trials 5 --iters 100
python plot_metrics.py --input results/sweep_C.csv \
    --outdir plots/sweep_C
```

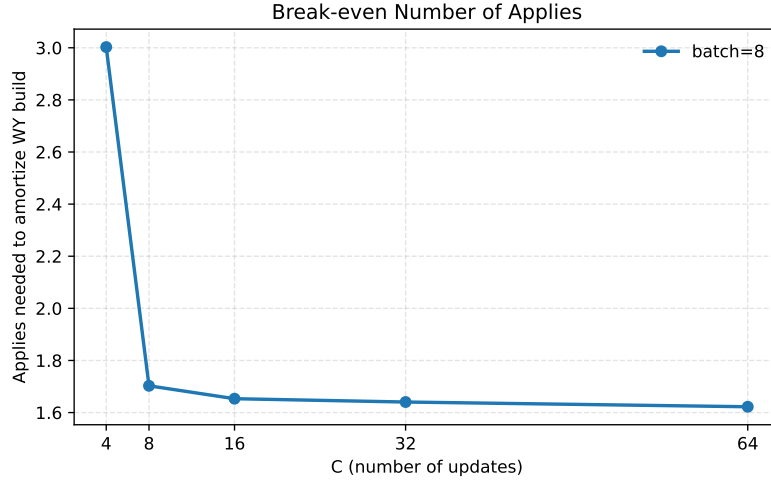


Figure 3: Break-even applies (number of reuses needed to amortise WY build cost). For $C \geq 8$, break-even is under 2 applies. At $C = 4$, it rises to ≈ 3 applies due to Tensor-Core under-saturation at small tile size.

A fixed random seed (`torch.manual_seed(42)`) is set at the start of each trial, with a per-case offset to ensure independent trials. Timing uses `time.perf_counter` with `torch.cuda.synchronize()` barriers and 10-iteration GPU warmup before measurement.

Code Availability

All benchmark code, scripts, and raw results are available at:
<https://github.com/faisalmemon/kimi-k3-householder>

References

- [1] Kimi Team. *Kimi Linear: Scaling Up Linear Recurrent Models*. arXiv:2510.26692, 2025.
- [2] C. Bischof and C. Van Loan. *The WY Representation for Products of Householder Matrices*. SIAM Journal on Scientific and Statistical Computing, 8(1):s2–s13, 1987.
- [3] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. *Basic Linear Algebra Subprograms for Fortran Usage*. ACM Transactions on Mathematical Software, 5(3):308–323, 1979.